

Fault- and Network-Agnostic Agreement

Erica Blum¹, Evan Hart¹, Vivek Malik¹, and Aidan Mokalla¹

Reed College, Portland, Oregon ericablum@reed.edu **

Abstract. Classical fault-tolerant consensus protocols are typically designed for a specific timing assumption (e.g. synchrony, partial synchrony or asynchrony) and fault model (e.g., crash or Byzantine). We generalize the notion of a network-agnostic protocol to protocols that are agnostic with respect to network and fault type, establishing tight conditions on feasibility for a natural agreement problem across $\binom{2 \times 3}{2} = 15$ *pairs* of classical models and comparing the costs and benefits of agnostic consensus with those of classical and mixed fault-tolerant protocols.

Keywords: Byzantine Agreement · Consensus · Network Agnostic.

1 Introduction

Consensus is a fundamental problem in distributed systems in which a set of devices must reach agreement on a shared output, despite possible faults in devices and/or the underlying network. Whether consensus is solvable depends on the details of the system model, including the number and type of faulty parties, timing assumptions on communication channels, and availability (or lack) of cryptographic tools. Over time, most work in this area has converged on a handful of classical fault models (crash, omission, and Byzantine) and timing assumptions (synchrony, partial synchrony, and asynchrony) that are simple enough to facilitate deep theoretical study yet still expressive enough to capture a wide variety of real-world conditions.

One limitation of some protocols designed for a specific classical model is that they may not have well-defined behavior if conditions at runtime are different than expected. In some cases, this is unavoidable: if the conditions at runtime run afoul of a known impossibility result, then we cannot hope for anything better. But what if we were simply unlucky, and the actual conditions at runtime were survivable by *some* protocol, just not by *our* protocol? *Network-agnostic* consensus (introduced by Blum, Katz, and Loss [8]) seeks to partially address this question by constructing Byzantine fault-tolerant (BFT) consensus protocols with well-defined security guarantees under two different timing assumptions. In other words, a network-agnostic protocol allows a system designer to hedge their bets regarding the underlying timing model. A growing line of work has expanded on the original results in various ways, including extensions to other problems like multiparty computation (MPC) [23, 5, 9], state machine

** Authors are listed in alphabetical order.

replication (SMR) [4], multidimensional approximate agreement [20], and convex agreement [16]; varying validity properties [14]; partially synchronous fall-back [26]; and black-box constructions [17].

In this work, we generalize network-agnostic protocols to [model-]agnostic protocols, in which the two ‘modes’ can differ in their fault type as well as the timing assumption. Fig. 1 gives a visual summary of our results.

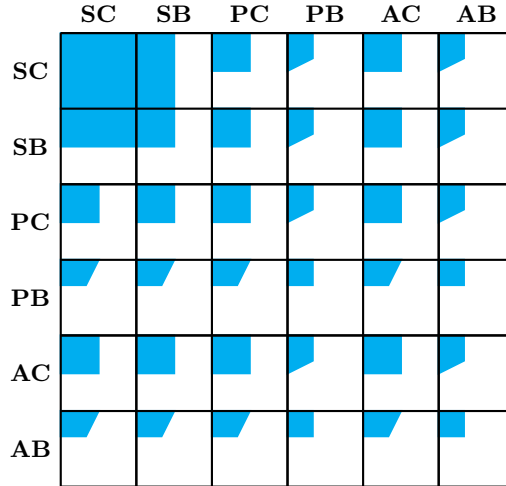


Fig. 1. Each cell corresponds to a model pair, with the shaded area representing the feasibility region for that model pair. A point $(t_x, t_y) \in [0, n]^2$ within the cell is colored blue if there is a fault-agnostic agreement protocol tolerating t_x faults in the column’s model, and t_y faults in the row’s model. Cells on the diagonal correspond to classical single-model results.

1.1 Additional Related Work

In the original network-agnostic model [8], the network may be asynchronous with t_a faults or synchronous with t_s faults; the same work shows a bound of $t_a + 2t_s < n$ for standard Byzantine agreement. The elegant work of Momose and Ren [26] goes above and beyond [8] by decoupling the thresholds for consistency and liveness and showing tight bounds for SMR with *partially synchronous* fall-back. Looking ahead, because we consider a slightly different agreement problem, our impossibility result does not follow trivially from theirs; we will revisit this point in more detail later, in Section 3.4.

A long line of work considers *mixed faults*, in which two or more kinds of faults may coexist in the same execution [13, 2, 22]. For example, [13] and [2] study mixed crash and Byzantine faults in the asynchronous and synchronous settings, respectively. Protocols with mixed fault tolerance offer a kind of agnostic security

(and vice versa), though neither appears to subsume the other. (We expand on this point in Section 3.4.)

Other prior work with distinct but related goals includes consensus in granular synchrony [21], in which timing assumptions vary by channel; the flexible BFT [25] protocol, which accommodates parties with varying beliefs about the underlying network and thresholds; and Sync HotStuff [3], which ensures liveness under synchrony but safety under weaker assumptions. The BFT-SMART protocol [7] can toggle between tolerating t_1 crash or t_2 Byzantine faults within a single (partially synchronous) execution when $t_1 < n/3$ and $t_2 < n/2$. By comparison, our agnostic protocols provide their guarantees automatically (without reconfiguration), but cannot switch between them within a single execution as BFT-SMART does.

2 Models, Definitions, and Background Results

Throughout, we consider n parties connected by a complete network of authenticated point-to-point channels. In any execution, an adversary may control a subset of *faulty* parties; the rest are *correct*. We consider two different *fault models* (i.e., assumptions on the behavior of faulty parties): crash and Byzantine. In both cases, an adversary can adaptively *corrupt* up to some fixed number of parties over the course of a protocol execution. In the crash fault model, the adversary can cause a corrupted party to crash. A crashed party does not execute any more protocol instructions, and does not send or receive messages. In the Byzantine fault model, the adversary can cause a corrupted party to deviate arbitrarily from the protocol. We refer to a party that is not Byzantine at any point during an execution as *benign*. (N.b. that all correct parties are benign, but not all benign parties are correct; instead, benign is a synonym for ‘never Byzantine.’)

We refer to a set of assumptions about the underlying communication network as a *network model*. For our purposes, the defining feature of a network model is its timing assumption: *synchrony*, *partial synchrony*, or *asynchrony*. In a synchronous network model, a message sent at time T must be delivered by time $T + \Delta$, where Δ is a fixed parameter known to all parties. In a partially synchronous network model, there exists a *global stabilization time* T_{GST} that is unknown to the correct parties such that messages sent at time T must be delivered before time $\max\{T_{\text{GST}}, T\} + \Delta$. In an asynchronous network model, a message sent by a correct party is eventually delivered, but no other assumptions are made on message timing. Protocols are analyzed in the presence of an adversary (for simplicity, the same one controlling the faulty parties) that can reorder and delay messages arbitrarily subject to the timing assumption(s).

Each party has a local clock. We assume benign parties’ clocks increase monotonically and are synchronized to the same global time under synchrony and after

GST under partial synchrony; otherwise (after GST and under asynchrony), the adversary may arbitrarily control the local clock of each party.¹

In this work, we are interested in mixing and matching fault models with timing models. To facilitate this, we define some terminology and notation to assist with describing, classifying, and comparing models. Our notation takes loose inspiration from the careful and elegant formalization of flavors of partial synchrony in Unifying partial synchrony [15].

Definition 1 (Models and Model Families). *A [system] model $M(t)$ is a set of assumptions defined by a model family M and a fault threshold $t : \mathbb{N} \rightarrow \mathbb{N}$. We say that a protocol Π tolerates a model $M(t)$ if Π is secure in the presence of an adversary that must obey the assumptions of model family M and can control up to t parties in a single execution. The model family specifies (at a minimum) the timing assumptions for the network and the fault model.*

We write $M_1(t) \subseteq M_2(u)$ if all executions satisfying the assumptions of $M_1(t)$ also satisfy the assumptions of $M_2(u)$. In this case, we sometimes say that M_1 is more restrictive (from the adversary’s perspective).

For our purposes, a model family M has two defining features: the network model and the fault model. (Since our assumptions about local clocks are closely linked to the network model, we consider them to be part of the network model.) Mixing and matching the three classical network models (synchrony, **p**artial synchrony, and **a**synchrony) with the two fault models (**c**rash and **b**yzantine) yields 6 concrete model families, which we refer to by the initials of the network model and the fault model: SC, SB, PC, PB, AC, and AB. Together, a model family and a fault threshold define a model.

In principle, a system model can vary in many more dimensions than just the fault model, network model, and number of faults. However, in this work, we will keep all other details of the system model fixed. In particular, we assume adaptive corruptions, that communication channels are authenticated (but not private), and the parties have established a public key infrastructure (PKI) for use with a digital signature scheme. Our adversary cannot perform after-the-fact removal, i.e., cannot “take back” a message that was sent by a non-faulty party, even if the adversary corrupts that party after the message was sent but before it is delivered.

Throughout, we use $\langle m \rangle_i$ as a shorthand for a message m accompanied by a valid signature by party p_i on m . Whenever a protocol expects a message of this form (e.g. “On receiving $\langle m \rangle_i$ from p_i , ...”), we implicitly assume that parties ignore any protocol message with an invalid signature. Since we sometimes have multiple protocols running simultaneously, we also implicitly assume that each message is associated with a unique protocol instance (say, using a tag) to avoid ambiguity and prevent the adversary from replaying messages out of context.

¹ We follow prior work on network-agnostic protocols in making the slightly non-standard assumption that parties always see the same clock “interface”, even when the network is asynchronous. We stress that the clock’s properties depend on the network and are not known to benign parties a priori.

Definition 2 (Model pairs and agnostic protocols). Given two models $M_1(t)$ and $M_2(u)$, the **model pair** $M_1(t) \cup M_2(u)$ is a special system model in which at runtime, the system is assumed to satisfy the assumptions of either M_1 or M_2 . The underlying model in a given execution is known to the adversary but not to the correct parties.

We say that protocol Π tolerates model pair $M_1(t) \cup M_2(u)$ if Π tolerates $M_1(t)$ and Π tolerates $M_2(u)$. When the specific fault thresholds t, u are irrelevant or clear from context, we sometimes omit them and simply write $M_1 \cup M_2$.

Restrictiveness extends to model pairs via the following rule: $M_1(t) \cup M_2(u) \subseteq M_3(v) \cup M_4(w)$ when $M_1(t) \subseteq M_3(v)$ and $M_2(u) \subseteq M_4(w)$, or when $M_1(t) \subseteq M_4(w)$ and $M_2(u) \subseteq M_3(v)$.

We now define our problem of interest, a variant of *binary agreement*. Informally, our goal is to achieve the usual properties of Byzantine or crash fault-tolerant agreement when run in a model with Byzantine or crash faults, respectively. Formally:

Definition 3 (Fault-agnostic binary agreement). Let Π be a protocol run among n parties $p_1, \dots, p_n$ where each p_i has a binary input $b_i \in \{0, 1\}$ and may eventually decide an irrevocable binary output $b'_i \in \{0, 1\}$. We say that Π solves fault-agnostic binary agreement if it guarantees all of the following (possibly except with probability negligible in a security parameter):

- **Consistency:** if p_i, p_j are benign parties that decide bits b'_i, b'_j respectively, then $b'_i = b'_j$.²
- **[Benign-Unanimous] Validity:** if all benign parties receive the same input b , then for any benign party p_i that decides a value $b'_i \in \{0, 1\}$, it holds that $b'_i = b$.³
- **Termination:** all correct parties eventually decide and terminate.

We additionally define non-triviality to be the property that at least one party decides.

We conclude the preliminaries with a brief review of relevant classical (single-model) results. Throughout, unless otherwise noted, results assume authenticated channels and public-key infrastructure (PKI). We limit our attention to uniform agreement with benign-unanimous validity in models with crash faults, and non-uniform agreement with strong validity in models with Byzantine faults.

In the synchronous-crash (SC) model, agreement is feasible for $t < n$ faults [24]. In the synchronous-Byzantine (SB) model, agreement is feasible if and only if $t < n/2$ (assuming PKI; otherwise only $t < n/3$). In the asynchronous-crash

² Readers might recognize this property as uniform agreement in the crash case and non-uniform agreement in the Byzantine case; see Appendix A for brief comments on the choice of definition.

³ In the presence of Byzantine faults, benign-unanimous validity is equivalent to the usual notion of validity for Byzantine agreement. See A for a comparison to typical notions of validity in the crash fault model.

(AC) model, the agreement protocol of Ben-Or shows feasibility for $t < n/2$ [6]; this immediately carries over to the partially synchronous-crash model (PC). A matching impossibility result in PC [19] shows that this threshold is tight in both models. Finally, in the partially synchronous-Byzantine (PB) model, Dwork, Lynch, and Stockmeyer show that agreement is impossible for $t \geq n/3$ [19] (and hence also impossible in the asynchronous-Byzantine model). In the asynchronous-Byzantine (AB) model, agreement is feasible if [10] and only if [11] $t < n/3$, with the caveat that randomization is necessary.

3 Technical Results

We begin by gathering a few low-hanging corollaries of results in prior work, starting with a simple corollary of the main theorems of Blum, Katz, and Loss [8].

Theorem 1 (Feasibility and impossibility for Byzantine agreement in $\text{SB} \cup \text{AB}$ [8, Theorems 3 & 4]). *There is a Byzantine agreement protocol tolerating $\text{SB}(t_{sb}) \cup \text{AB}(t_{ab})$ if and only if $3t_{ab} < n$ and $2t_{sb} + t_{ab} < n$.*

Recall that our target definition (Def. 3) is not identical to the standard Byzantine agreement definition (which requires non-uniform agreement and strong validity), but the difference appears only in the crash case. Therefore, the extension to our definition is immediate for $\text{SB}(t_{sb}) \cup \text{AB}(t_{ab})$ (and indeed any model pair with exclusively Byzantine faults). Stated concretely:

Corollary 1 (Feasibility and impossibility for fault-agnostic binary agreement $\text{SB} \cup \text{AB}$). *There is a fault-agnostic binary agreement protocol tolerating model pair $\text{SB}(t_{sb}) \cup \text{AB}(t_{ab})$ if and only if $3t_{ab} < n$ and $2t_{sb} + t_{ab} < n$.*

Next, we observe that the original impossibility theorem can be strengthened as follows:

Corollary 2 (Impossibility for Byzantine agreement in $\text{SC} \cup \text{PB}$). *There is no (non-uniform and strongly valid) Byzantine agreement protocol tolerating $\text{SC}(t_c) \cup \text{PB}(t_b)$ if $3t_{pb} \geq n$ or if $2t_{sc} + t_{pb} \geq n$.*

Proof. $3t_{pb} < n$ is a standard bound for the Byzantine, partially synchronous setting. Inspecting the proof of [8, Theorem 4], we note that the Byzantine parties in their experiment immediately crash in the synchronous case, allowing us to strengthen the result to $2t_{sc} + t_{ab} \geq n$ in $\text{SC} \cup \text{AB}$. Furthermore, partial synchrony suffices for the adversary's attack, so we can strengthen it again to $2t_{sc} + t_{pb} \geq n$ in $\text{SC} \cup \text{PB}$. Thus, there is no non-uniform and strongly valid agreement protocol that tolerates $2t_{sc} + t_{pb} \geq n$ in $\text{SC} \cup \text{PB}$. $\square$

It tempting to think that this impossibility result immediately implies impossibility for fault-tolerant binary agreement in $\text{SC} \cup \text{PB}$ for the same bounds, but unfortunately it does not. The two definitions are equivalent in the presence of Byzantine faults, but in the crash case, impossibility of non-uniform and

strongly valid agreement in model $M(t)$ does *not* imply impossibility of fault-tolerant binary agreement in model $M(t)$. (As a simple counterexample, note that non-uniform and strongly valid agreement is impossible for $t \geq n/2$ crash faults in synchrony, but fault-tolerant binary agreement is feasible for $t < n$ crash faults in synchrony.) Fortunately, we can still prove the desired result using a standard partitioning argument.

Corollary 3 (Impossibility for $\text{SC} \cup \text{PB}$). *There is no fault-agnostic binary agreement protocol tolerating $\text{SC}(t_c) \cup \text{PB}(t_b)$ if $3t_{pb} \geq n$ or if $2t_{sc} + t_{pb} \geq n$.*

Proof. Suppose (towards a contradiction) that Π is a fault-agnostic binary agreement protocol tolerating $\text{SC}(t_c) \cup \text{PB}(t_b)$ such that $3t_{pb} \geq n$ or $2t_{sc} + t_{pb} \geq n$. The case $3t_{pb} \geq n$ is precluded by the classical result for Byzantine agreement in partial synchrony, so the interesting case is when $2t_{sc} + t_{pb} \geq n$.

Divide the parties into nonempty sets S_0 , S_1 , and A of size (up to) t_{sc} , t_{sc} , and t_{pb} , respectively. Consider an execution in $\text{PB}(t_{pb})$ in which all parties in A are Byzantine and the remaining parties are benign, and furthermore parties in S_0 have input 0 and parties in S_1 have input 1. The adversary delays all messages between S_0 and S_1 , but delivers all other messages within time Δ . (If GST occurs before at least one party in S_0 and one party in S_1 have output, say that the adversary gives up.) Meanwhile, the corrupted parties behave as if they had input 0 and have received nothing from parties in S_1 when interacting with parties in S_0 , and behave as if they had input 1 and have received nothing from parties in S_0 when interacting with parties in S_1 .

Now consider the view of a party in S_0 in an execution in $\text{SC}(t_{sc})$ where *all* parties receive input 0 and the parties in S_1 crash immediately at the beginning of the execution. By liveness, in a given execution, there must be some time T_0 by which the first party in S_0 outputs, and by validity, that party's output must be 0. Furthermore, the party's view in this execution is indistinguishable from its view in the original partially synchronous execution (as long as GST occurs after T_0), so it must output 0 in that execution, too. But by a symmetric argument, there is some time T_1 in the originally partially synchronous execution by which some party in S_1 eventually outputs 1 (as long as GST occurs after T_1). Thus, there are values of GST for which consistency is violated in the partially synchronous case, and we reach a contradiction. $\square$

Next, we turn our attention to an impossibility result in the $\text{SC} \cup \text{PC}$ model pair, in which the network may be synchronous or partially synchronous and all faults are crash faults. In contrast to the previous result, there is no explicit tradeoff between the two thresholds, but we do give something up: tolerating partial synchrony even for 0 faults lowers the optimal fault tolerance in the synchronous mode from $t < n$ to $t_s < n/2$.

Corollary 4 (Impossibility for $\text{SC} \cup \text{PC}$). *There is no fault-agnostic binary agreement protocol tolerating $\text{SC}(t_s) \cup \text{PC}(t_p)$ if $2t_s \geq n$ or if $2t_p \geq n$.*

Proof. Impossibility for $2t_p \geq n$ is inherited from the classical impossibility result for crash faults in partial synchrony, so the interesting case is the synchronous

mode. Suppose a fault-agnostic binary agreement protocol Π tolerates t_s crashes in synchrony, where $2t_s \geq n$. We show that Π cannot tolerate partial synchrony whatsoever, even with 0 crashes.

Consider an experiment in which the adversary partitions the n parties into a set S_0 of $\lceil n/2 \rceil$ parties and a set S_1 of $\lfloor n/2 \rfloor$ parties. All parties in S_0 receive input 0 and all parties in S_1 receive input 1. No party crashes, and any messages within S_0 and within S_1 are delivered within time Δ , but (for now) all communication between S_0 and S_1 is delayed.

The views of parties in S_1 in this experiment are identically distributed to their views in a *synchronous* execution of Π in which all parties receive input 1 but all parties in S_0 crash immediately, which Π must tolerate since $\lceil n/2 \rceil \leq t_s$ if $2t_s \geq n$. Thus, by t_s -validity of Π , parties in S_1 must, if they decide, decide 1. Furthermore, by t_s -termination of Π , there must be a time T such that all parties in S_1 have decided 1 by time T except with negligible probability.

Similarly, the views of parties in S_0 are identically distributed to their views in an execution of Π in which all parties receive input 0 but all parties in S_1 immediately crash, which Π must tolerate since $\lfloor n/2 \rfloor \leq t_s$ if $2t_s \geq n$. As such, by t_s -validity and t_s -termination, there must be some time T' such that all parties in S_0 must decide 0 by some time T' except with negligible probability.

Finally, consider a *partially synchronous* execution of Π in which no parties crash, communication within S_0 and within S_1 is bounded by Δ , but communication between S_0 and S_1 is delayed until time $\max\{T, T'\} + 1$. If the global stabilization time occurs after $\max\{T, T'\} + 1$, then the views of all parties in this execution over the time interval $\max\{T, T'\} + 1$ are identically distributed to the views of all parties over the same interval in the original experiment, because all parties terminate by the time any communication between the parties can reveal any difference between the experiment and the execution. This implies that parties in S_0 decide 0 while parties in S_1 decide 1, so consistency is violated. $\square$

As a second warmup (and our first non-trivial feasibility result), we consider one of the simplest and most natural model pairs: crash faults and Byzantine faults in synchrony. After the tradeoffs and costs of the previous section, we might be pleasantly surprised to find that uniform agreement for $t_c < n$ crash faults is “free” for synchronous Byzantine agreement protocols (or vice versa). This makes this model pair something of an outlier in the landscape of agnostic agreement (though “free” or “almost-free” agnostic security has been observed previously for other problems, e.g., in MPC and DKG [5, 23]).

We show feasibility for this model pair via a minor twist on a building block from the original network-agnostic paper (Π_{SBA} , Fig. 2 [8]), which is itself a reasonably straightforward modification of the canonical reduction from agreement to broadcast under synchrony for $t < n/2$ Byzantine faults, in which parties each broadcast their input (say, using Dolev-Strong broadcast [18]) and then outputs the majority value from among the outputs of the n broadcasts.

The twist in our case appears in the output determination step. In the agreement protocol of [8], parties output $\perp$ in the event that too few broadcasts terminate with non- $\perp$ output. This permits a weak form of validity even when

the network is asynchronous. In model pair $\text{SC} \cup \text{SB}$, the network is always synchronous, so we can relax this condition to tolerate $t_c \geq n/2$ crash faults.

Algorithm 1: $\Pi_{\text{SCUSB}}^{t_c}$: a fault-agnostic agreement protocol for model pair $\text{SB}(t_b) \cup \text{SC}(t_c)$, described from the perspective of party i with input $v_i \in \{0, 1\}$, and using Π_{DS} of [8] as a subprotocol.

Round 1 . . . $n - 1$:

 Participate in n parallel executions of Π_{DS} with senders $p_1, \dots, p_n$, respectively, using v_i as the input to the i^{th} broadcast.

After round $n - 1$:

 Let v_{maj} be the most frequent non- $\perp$ value among $v'_1, \dots, v'_n$, where v'_j denotes the output of the j^{th} execution of Π_{DS} (break ties with $v_{maj} = 0$).
 Output v_{maj} and terminate.

We now prove security of $\Pi_{\text{SCUSB}}^{t_c}$, taking the following lemma of [8] as a starting point.

Lemma 1 ([8, Lemma 1]). *For any fixed $t < n$, broadcast protocol Π_{DS} (Fig. 1, [8]) guarantees the following in synchrony with at most t Byzantine faults:*

- (Validity) *If a correct sender inputs v , every correct party outputs v .*
- (Consistency) *There exists $v \in \{0, 1, \perp\}$ s.t. each correct party outputs v .*

Theorem 2 (Feasibility of fault-agnostic binary agreement in $\text{SC} \cup \text{SB}$). *Fix n and t_c, t_b such that $t_c < n$ and $2t_b < n$. $\Pi_{\text{SCUSB}}^{t_c}$ is an agnostic protocol for fault-agnostic binary agreement in the model pair $\text{SB}(t_b) \cup \text{SC}(t_c)$.*

Proof. We begin by proving that $\Pi_{\text{SCUSB}}^{t_c}$ is consistent in each model. Because Π_{DS} is consistent and valid for any $t < n$, it retains its properties in both $\text{SB}(t_b)$ and $\text{SC}(t_c)$. In model $\text{SB}(t_b)$, all correct parties observe the same broadcast outputs and hence output the same value, so $\Pi_{\text{SCUSB}}^{t_c}$ is consistent in $\text{SB}(t_b)$. Meanwhile, in $\text{SC}(t_c)$, consistency holds automatically for correct parties, so it only remains to argue that no p_i, p_j can output different values even if one or both are crash faulty. If a crash-faulty party decides, it has behaved indistinguishably from a correct party until that point, so by the same reasoning as above, it sees the same broadcast outputs as any other party.

Next we prove validity, starting with $\text{SB}(t_b)$. In this case, the set of benign parties is exactly the set of correct parties, so suppose that all correct parties receive input b . Then strictly more than half of the broadcast instances have a correct sender, so by validity of Π_{DS} , all parties end up with strictly more than $n/2$ copies of b among the broadcast outputs. Thus, they all decide b . Finally, we show $\Pi_{\text{SCUSB}}^{t_c}$ obtains validity in $\text{SC}(t_c)$. If every party receives input b then every correct party's output sequence contains at least one copy of b (because there is at least one benign sender and thus one successful instance of DS) and

no copies of $1 - b$ (because no parties equivocate). Thus, the majority non- $\perp$ output is always b , so all correct parties output b . $\square$

The matching impossibility result follows immediately from the single-model bounds; we state it here for completeness and defer the proof to the appendix.

Theorem 3 (Impossibility of fault-agnostic binary agreement in $\text{SC} \cup \text{SB}$). *For any n and $t_c, t_b \leq n$, if $t_c \geq n$ or $2t_b \geq n$, there is no n -party protocol for fault-agnostic binary agreement that tolerates $\text{SC}(t_c) \cup \text{SB}(t_b)$.*

3.1 Crash and Byzantine faults in asynchrony

We now make our first major foray into asynchrony with the $\text{AC} \cup \text{AB}$ model pair. The central result of this section is a protocol $\Pi_{\text{AC} \cup \text{AB}}^{t_c, r_{\max}}$ (shown in Alg. 2) that, for any fixed thresholds t_c, t_b such that $3t_b < n$ and $2t_c + t_b < n$, tolerates $\text{AC}(t_c) \cup \text{AB}(t_b)$ except with negligible probability in the protocol parameter $r_{\max}$.

Our construction is a variant of (Shi’s variant of) the randomized asynchronous agreement protocol of Cachin, Kursawe, and Shvartsman [28, 12]. At a high level, the protocol proceeds in asynchronous (event-based) rounds, using a coin-flip primitive (denoted `coinflip`) to break deadlock and ensure eventual termination (with high probability). The coin-flip primitive takes as input an integer R and is assumed to have the following properties: before $n - t_c$ distinct parties call `coinflip`(R), the adversary learns nothing about the value of `coinflip`(R); once $n - t_c$ distinct parties call `coinflip`(R), all correct parties are eventually notified of its value; and the result $c \leftarrow \{0, 1\}$ is uniform and independent of everything that happens before the coin is flipped.⁴

Each protocol message carries a party’s signature, a round number, a bit, a title (either *prevote*, *vote*, or *terminate*), and a justification (a subset of at least $n - t_c$ distinct parties, specified by a set of indices $J \subseteq [n]$). During the protocol, when a party p_i receives a new message, it forwards it to all other parties. Building on this, we say p_i *certifies* a message m when it determines that m is consistent with the behavior of a benign party, given the other messages it has received. More formally, p_i certifies message m when one of these conditions holds:

- m is marked as round 0.
- m is marked as a vote (resp. prevote) from round R , and from each party p_j in m ’s justification, p_i has certified a prevote (resp. vote) from round R (resp. $R - 1$), signed by p_j , such that the protocol would send the message m if it received that set of prevotes (resp. votes).
- m is marked as a termination message from round R , and from each party p_j in m ’s justification, p_i has certified a vote from round R signed by p_j for m ’s bit.

⁴ We can instantiate such a coin-flip with the coin-flip of [12], which supports a threshold of k requests, secure under t arbitrary faults, for any $k \in (t, n - t]$. In our case, we have $t_b < n - t_c \leq n - t_b$, so we can use their coin-flip as-is with threshold $n - t_c$.

As a final special case, a certified termination message for round R qualifies as a certified round- $(r + 1)$ prevote and a certified round- r vote for all rounds $r \geq R$. (In other words, a certified termination message is interpreted as a prevote and vote in future rounds.) We say a set of messages $S = \{m_1, m_2, \dots\}$ is certified if each message in S is certified.

Because the definition of a certified message is recursive, each time p_i certifies a newly received message, it must then check outstanding (not yet certified) messages from rounds $r \geq R$ to check if it should certify them as a result. Importantly, if a Byzantine party equivocated, there may be multiple messages with the same party, round, and title, but different bits; if *at least one* message from each party cited by m supports m , then m is certified.

Finally, a message is *properly justified (PJ)* if there exists a benign party that has certified it, and a set of votes S is *unanimous for v* if every vote in S is for value v ; if the value v is clear from context (or not relevant), we simply say S is unanimous.

With the preliminaries out of the way, we can describe and analyze the full protocol, $\Pi_{\text{ACUAB}}^{t_c, r_{\max}}$ (Alg. 2). We begin with some basic properties of $\Pi_{\text{ACUAB}}^{t_c, r_{\max}}$, assuming model pair $\text{AC}(t_c) \cup \text{AB}(t_b)$ throughout.

Lemma 2. (*Credibility*) *If a correct party sends a message, all correct parties eventually certify it (or terminate first).*

Proof. The correct party p_i that sends a message does so on the basis of some messages that p_i certifies, which it forwards when it receives them; p_i certifies these on the basis of some other p_i -certified messages which p_i also receives and forwards, and so on. All of these messages eventually arrive at all other non-terminating correct parties, so they all eventually certify all of them. $\square$

Lemma 3. (*AC validity*) *If all parties receive input v , if at most t_c parties are crash faulty and all others are correct, then all correct parties output v .*

Proof. There are $\geq n - t_c$ votes from correct parties for v sent in round-0, and no votes for $1 - v$ since all parties are benign. All correct parties receive and certify these and so cast a prevote in round 1 for v , and again no party casts a prevote for $1 - v$. All correct parties certify at least $n - t_c$ round-1 prevotes for v , so cast a round-1 vote for v . So all correct parties certify $n - t_c$ votes for v in round 2. Then they all decide v at the start of round 2 and ignore the coin flip. $\square$

Lemma 4. (*AB validity*) *Fix $t_b \leq t_c$ such that $t_b < n - 2t_c$. If at most t_b parties are faulty, and all correct parties receive input v , then all correct parties decide v .*

Proof. Every correct party sends a round-0 vote for v , so all correct parties receive enough round-0 votes to prevote in round 1. Of the $n - t_c > t_b + t_c \geq 2t_b$ round-0 votes signed by distinct parties received in round 1, more than half of them are for v (at most t_b , less than half, are from faulty parties, so are possibly not for v). Thus, all correct parties prevote for v in round 1. Furthermore, no

Algorithm 2: $\Pi_{\text{ACUAB}}^{t_c, r_{\max}}$, from the perspective of party p_i with input $v_i \in \{0, 1\}$

Throughout, upon receiving a vote with title t , bit b , and round r signed by p_i :

If this is the first vote received for t , b , r signed by p_i , forward it to all parties.

Round 0.

└ Multicast $\langle \text{vote}, r = 0, v_i \rangle$.

Round 1.

└ **Upon** certifying $n - t_c$ round-0 votes signed by $n - t_c$ distinct parties

└ $V_0 \subset [n]$:

└ Let b be the majority bit among those votes (breaking a tie with 0).

└ Multicast $\langle \text{prevote}, r = 1, b, V_0 \rangle_i$.

└ **Upon** certifying a set U'_1 of $n - t_c$ round-1 prevotes signed by $n - t_c$ distinct parties $U_1 \subset [n]$:

└ If U'_1 is unanimous for some $b \neq \perp$, multicast $\langle \text{vote}, r = 1, b, U_1 \rangle_i$;

└ otherwise, multicast $\langle \text{vote}, r = 1, \perp, U_1 \rangle_i$.

Round $R = 2, \dots, r_{\max}$.

└ Complete all earlier rounds first.

└ **Upon** certifying a set V'_{R-1} of $n - t_c$ round- $(R - 1)$ votes signed by $n - t_c$ distinct parties $V_{R-1} \subset [n]$:

└ $c \leftarrow \text{coinflip}(R)$.

└ If V'_{R-1} contains a vote for $b \neq \perp$, multicast $\langle \text{prevote}, r = R, b, V_{R-1} \rangle_i$;

└ otherwise, multicast $\langle \text{prevote}, r = R, c, V_{R-1} \rangle_i$.

└ **Upon** certifying a set U'_R of $n - t_c$ round- R prevotes signed by $n - t_c$ distinct parties $U_R \subset [n]$:

└ If U'_R is unanimous for some $b \neq \perp$, multicast $\langle \text{vote}, r = 1, b, U_R \rangle_i$;

└ otherwise, multicast $\langle \text{vote}, r = 1, \perp, U_R \rangle_i$.

Upon there existing a round $R \geq 1$ and a bit $b \in \{0, 1\}$ such that there are at least $n - t_c$ certified votes from round R on bit b , signed by a set T_R of $n - t_c$ distinct parties:

└ Complete execution as above for all rounds up to and including R .

└ Multicast $\langle \text{terminate}, r = R, b, T_R \rangle_i$.

└ Output b and terminate.

party will certify a round-1 prevote for $1 - v$ because a prevote for $1 - v$ requires $> t_b$ parties to have prevoted for $1 - v$ in round 0. Thus, the only prevotes that can be certified are for v , and eventually each correct party certifies $n - t_c$ prevotes, and votes for v in round 1. Again, no party can vote for $1 - v$ or $\perp$ because each of those votes requires at least one round-1 prevote for $1 - v$, but no correct party certifies any such vote. As such, at the start of round 2 each correct party will only certify votes for v , and will receive enough of these to decide v at the start of round 2. $\square$

Let F be the first round for which any correct party eventually certifies a termination message marked for round F . The next lemma shows consistency through round F .

Lemma 5. (*Intra-round consistency*) *If $\Pi_{AC \cup AB}^{t_c, r_{max}}$ is run in model pair $AC(t_c) \cup AB(t_b)$ where $t_c < n/2$ and $t_b < n - 2t_c$, then the following hold for any R with $1 \leq R \leq F$:*

- (a) *Every correct party prevotes and votes exactly once in round R .*
- (b) *There is not both a PJ round- R vote for 0 and a PJ round- R vote for 1.*

Proof. We induct on R . In round 1, every correct party receives at least $n - t_c$ round-0 votes, all of which are immediately certified, so every correct party prevotes exactly once. Since at least $n - t_c$ parties are correct, every correct party receives (and certifies, by 2) enough prevotes to vote, and does so once. Thus, (a) holds in round 1. Next, if there is a PJ round-1 vote for 0 that cites a set of $n - t_c$ parties and a PJ round-1 vote for 1 that cites a set of $n - t_c$ parties, these two sets intersect in at least $n - 2t_c > t_b$ parties, so some benign party is in the intersection. However, no benign party prevotes for both 0 and 1, by (a). Thus, (b) holds for $R = 1$.

Now let $R > 1$ and suppose (a) and (b) hold for round $R - 1$. Again, each correct party prevotes at least once, since it eventually certifies enough votes from correct parties. Note that this relies on a correct party not terminating, and none does at this round: terminating based on round- F votes only happens after round F , while terminating based on votes from a round $< F$ would lead to a termination message being sent and certified unless all correct parties terminate in which case no round- F message would be PJ. Correct parties do not prevote twice, because that would require PJ round- $(R - 1)$ votes for both 0 and 1, which cannot exist by the inductive hypothesis. This shows point (a). Point (b) holds by the same reasoning as given above for round 1. $\square$

The arguments above do not work in rounds $\geq F + 1$. This is because termination messages are only justified by votes from one round, but can be used in later rounds as both votes and prevotes; as such, the overlapping argument above does not hold. The following lemma shows, however, that in rounds after F the behavior of $\Pi_{AC \cup AB}^{t_c, r_{max}}$ is even more constrained.

Lemma 6. (*Consistency*) *In an execution of $\Pi_{AC \cup AB}^{t_c, r_{max}}$ in which any correct party certifies a termination message m for round F , all correct parties output b by the end of round $F + 2$.*

Proof. First, note that round- F votes cannot justify both a termination message for 0 and for 1, by Lemma 5, which holds in round F . Next, a termination message must be justified by at least $n - t_c$ round- F votes, which overlaps with any set of $n - t_c$ round- F votes by at least $n - 2t_c > t_b$ votes; thus, every correct and yet-to-terminate party certifies, at the start of round $F + 1$, at least one vote from a benign party cited by m . Since benign parties do not equivocate, such a vote must be for b . As such, each correct party casts a prevote (possibly through

a termination message) for b in round $F + 1$. Also, no correct party certifies any prevote for $1 - b$ since justifying one necessitates one of

- A vote for $1 - b$. This is impossible by Lemma 5.
- $n - t_c$ PJ $\perp$ votes from distinct parties. This is impossible because there are at least $n - t_c - t_b$ benign parties that vote for b , so at most $n - (n - t_c - t_b) < n - t_c$ votes for $\perp$.

It follows that all correct parties vote for b in round $F + 1$ (possibly through a termination message), so all remaining correct parties output b at the start of round $F + 2$. $\square$

It remains to show that the protocol terminates with high probability. The proof follows very similarly to the proof of termination in [28], so we state the key result here and include a detailed proof in the appendix for completeness.

Lemma 7. $\Pi_{AC \cup AB}^{t_c, r_{\max}}$ terminates with probability at least $1 - 2^{-\Omega(r_{\max})}$.

Theorem 4 (Feasibility for $AC \cup AB$). *There is a fault-agnostic binary agreement protocol tolerating $AC(t_c) \cup AB(t_b)$ if $3t_b < n$ and $2t_c + t_b < n$.*

Proof. Fix t_c, t_b with $3t_b < n$ and $2t_c + t_b < n$. Then we claim $\Pi_{AC \cup AB}^{\max(t_c, t_b), r_{\max}}$ obtains binary agreement if at most t_c parties crash and other parties are correct, or at most t_b parties are Byzantine and other parties are correct. As proof, notice that at most $t_b \leq \max(t_c, t_b)$ parties can be faulty, and $t_b < n - 2\max\{t_c, t_b\}$ whether $\max\{t_c, t_b\}$ is equal to t_c or t_b . Thus, the preconditions of all of the above lemmas are satisfied, and so $\Pi_{AC \cup AB}^{t_c, r_{\max}}$ is consistent, valid, and terminating (except with probability $2^{-\Omega(r_{\max})}$). $\square$

3.2 Feasibility and Impossibility for $SB \cup AC$

We now briefly turn our attention to the model pair $SB \cup AC$ and its corresponding construction, $\Pi_{SB \cup AC}^{t_c}$. This protocol, described below, is constructed as a wrapper around a building block which we call Π_{VAC} (“valid asynchronous consensus”). Π_{VAC} is very similar to $\Pi_{AC \cup AB}^h$, letting $h = \lceil n/2 \rceil - 1$ be the largest integer less than $n/2$, with the following minor modifications:

- Instead of just waiting for $n - h$ PJ prevotes/votes before moving on, party p_i waits for $n - h$ PJ prevotes/votes *and* Δ time (on its local clock) since it last prevoted/voted before moving on (lines 1, 2, 4, 5 of Algorithm 3).
- In round $r \geq 1$, party p_i instead votes for a bit b if it certifies $n - h$ round- r prevotes for b , rather than unanimity (lines 3, 6 of Algorithm 3).

Full pseudocode is included in Appendix B.3 for completeness.

Given Π_{VAC} , it is a relatively short leap to the full protocol, $\Pi_{SB \cup AC}^{t_c}$. In the first phase, each party acts as the sender for an instance of Byzantine broadcast (say, Dolev-Strong). After all broadcasts have terminated (or timed out), parties run Π_{VAC} using the majority non- $\perp$ output of the broadcasts as their input. The proofs follow very similarly to those already seen, so we state the main result here and defer the full pseudocode and proof to the appendix.

Theorem 5 (Feasibility and impossibility for $SB \cup AC$). *There is a fault-agnostic binary agreement protocol tolerating $SB(t_{sb}) \cup AC(t_{ac})$ if and only if $2t_{sb} < n$ and $2t_{ac} < n$.*

3.3 Everything Else

We now have everything we need to resolve all remaining model pairs, thus filling in the rest of Fig. 1.

Theorem 6 (Trapezoid Pairings). *For each model pair of form $SX(t_1) \cup YB(t_2)$, where $X \in \{C, B\}$ and $Y \in \{P, A\}$, there is a fault-agnostic binary agreement protocol tolerating that model pair if and only if $3t_2 < n$ and $2t_1 + t_2 < n$.*

Proof. Impossibility holds by Theorem 3 and the fact that all of these models are at most as restrictive as $SC(t_1) \cup PB(t_2)$. Feasibility holds by Theorem 1 and the fact that all of these models are at least as restrictive as $SB(t_1) \cup AB(t_2)$. $\square$

Theorem 7 (More Trapezoid Pairings). *For each model pair of form $XC(t_1) \cup YB(t_2)$, where $X, Y \in \{P, A\}$, there is a fault-agnostic binary agreement protocol tolerating that model pair if and only if $3t_2 < n$ and $2t_1 + t_2 < n$.*

Proof. Impossibility holds by Theorem 3 and the fact that all of these models are at most as restrictive as $SC(t_1) \cup PB(t_2)$. Feasibility holds by Theorem 4 and the fact that all of these models are at least as restrictive as $AC(t_1) \cup AB(t_2)$. $\square$

Theorem 8 ($n/2$ Square Pairings). *For each model pair of form $SX(t_1) \cup YC(t_2)$, where $X \in \{C, B\}$ and $Y \in \{P, A\}$, there is a fault-agnostic binary agreement protocol tolerating that model pair if and only if $2t_1 < n$ and $2t_2 < n$.*

Proof. Impossibility holds by Theorem 4 and the fact that all of these models are at most as restrictive as $SC(t_1) \cup PC(t_2)$. Feasibility holds by Theorem 4 and the fact that all of these models are at least as restrictive as $AC(t_1) \cup AB(t_2)$. $\square$

Theorem 9 (Trivial $n/2$ Square Pairing). *There is a protocol tolerating model pair $PC(t_1) \cup AC(t_2)$ if and only if $2t_1 < n$ and $2t_2 < n$.*

Proof. Feasibility for these thresholds can be achieved by an off-the-shelf protocol that tolerates $t_1 = t_2 = t < n/2$ crash faults in asynchrony. Impossibility is immediately inherited from impossibility results for each network in the pair. $\square$

Theorem 10 (Trivial $n/3$ Square Pairing). *There is a protocol tolerating model pair $PB(t_1) \cup AB(t_2)$ if and only if $3t_1 < n$ and $3t_2 < n$.*

Proof. Feasibility is achieved by running any protocol that tolerates $t_1 = t_2 = t < n/3$ Byzantine faults in asynchrony. Impossibility is inherited from impossibility results for each network in the pair. $\square$

3.4 Comparisons with Related Work

Agnostic vs. Mixed Fault Tolerance. In this work, we are mainly interested in a setting in which fault types do not mix within a single execution (except to the extent that, e.g., a Byzantine adversary can simulate a crash faulty party). Still, our equivalent protocols naturally achieve a mixed fault tolerance.

Remark 1. $\Pi_{ACUAB}^{t_c, r_{\max}}$ can tolerate t_c crashes and t_b Byzantine faults simultaneously when $2t_c + 3t_b < n$.

Proof. Every party will eventually certify at least $n - (t_c + t_b)$ messages, because there are at least that many correct parties. Also, there are at most $t_b < n - 2(t_c + t_b)$ Byzantine faults when $2t_c + 3t_b < n$. Thus all the lemmas pertaining to Π_{ACUAB} continue to apply.

This bound is known to be optimal in asynchrony and partial synchrony, so our protocol achieves optimal mixed-fault tolerance.

Remark 2. $\Pi_{SCUSB}^{t_c}$ can tolerate t_c crashes and t_b Byzantine faults simultaneously when $t_c + 2t_b < n$.

Proof. Due to unconditional consistency of the Dolev-Strong protocol in synchrony, consistency is automatic. Termination is also guaranteed because all parties stop executing after $(n + 1) \cdot \Delta$ time. If all benign parties receive input v then: by validity of DS, at least $n - t_c - t_b$ of the outputs received by a correct party from the instances of DS broadcast will be for v , and at most t_b will be for $1 - v$, so since $n - t_c - t_b > t_b$ it follows that every correct party will decide v .

[1] show that $t_c + 2t_b < n$ is optimal for mixed-fault tolerance in synchrony, so our protocol matches theirs in mixed-fault tolerance, with the caveat that our protocol is for binary rather than multivalued agreement.

The minimum guarantees provided by agnostic protocols are strictly weaker than those of agnostic protocols, because they apply only in the ‘extreme’ executions with all Byzantine or all crash faults (although by happy coincidence, our agnostic protocol also naturally tolerates mixed faults). Interestingly, this difference in definitions does not seem to ‘buy’ any feasibility in the synchronous model, where both agnostic protocols and mixed fault models face the same $t_c + 2t_b < n$ bound. Thus, our agnostic protocol has no notable advantage over a synchronous mixed fault protocol (e.g. the protocol of [1]). By contrast, in the partially synchronous and asynchronous models, the optimal fault tolerance for mixed fault protocols is $2t_c + 3t_b \geq n$, compared to $(3t_b < n \text{ and } 2t_c + t_b < n)$. Thus, in this case, the relative rigidity of an agnostic protocol allows it to make a more aggressive tradeoff.

Multi-Threshold BFT. To our knowledge, Momose and Ren initiated the study of synchronous BFT SMR with partially synchronous fallback (rephrased in our terminology, SMR in $\text{SB} \cup \text{PB}$) [26]. Comparing our results, we recover the same $2t_s + t_p$ bound for agreement in $\text{SB} \cup \text{PB}$ that Momose and Ren previously observed for SMR [26]. This is not especially surprising, but we remark that our bound for agreement matches their bound for SMR, which suggests that *provable* agreement is feasible for the same parameters as ‘plain’ agreement in $\text{SB} \cup \text{PB}$. It seems plausible that agnostic agreement protocols for those model pairs could be constructed from the SMR protocols of [26] (possibly even in a black-box way), but in this work we have opted to construct them directly.⁵

⁵ To our knowledge, black-box reductions between consensus primitives are an open question for agnostic protocols.

References

1. Abraham, I., Ben-David, N., Yandamuri, S.: Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In: Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing. pp. 381–391 (2022)
2. Abraham, I., Dolev, D., Eyal, I., Halpern, J.Y.: Distributed computing meets game theory: Robust mechanisms for rational secret sharing and multiparty computation. In: Proceedings of the 40th ACM Symposium on Principles of Distributed Computing. pp. 81–91 (2021)
3. Abraham, I., Malkhi, D., Nayak, K., Ren, L., Yin, M.: Sync hot-stuff: Simple and practical synchronous state machine replication. In: 2020 IEEE Symposium on Security and Privacy (SP). pp. 106–118 (2020). <https://doi.org/10.1109/SP40000.2020.00044>
4. Alexandru, A.B., Blum, E., Katz, J., Loss, J.: State machine replication under changing network conditions. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part I. LNCS, vol. 13791, pp. 681–710. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22963-3_23
5. Bacho, R., Collins, D., Liu-Zhang, C.D., Loss, J.: Network-agnostic security comes (almost) for free in DKG and MPC. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part I. LNCS, vol. 14081, pp. 71–106. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38557-5_3
6. Ben-Or, M.: Another advantage of free choice: Completely asynchronous agreement protocols. In: Proceedings of the 2nd Annual ACM Symposium on Principles of Distributed Computing. pp. 27–30 (1983)
7. Bessani, A., Sousa, J.a., Alchieri, E.E.P.: State machine replication for the masses with bft-smart. In: Proceedings of the 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. p. 355–362. DSN ’14, IEEE Computer Society, USA (2014). <https://doi.org/10.1109/DSN.2014.43>
8. Blum, E., Katz, J., Loss, J.: Synchronous consensus with optimal asynchronous fallback guarantees. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019, Part I. LNCS, vol. 11891, pp. 131–150. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-36030-6_6
9. Blum, E., Zhang, C.D.L., Loss, J.: Always have a backup plan: Fully secure synchronous MPC with asynchronous fallback. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 707–731. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_25
10. Bracha, G.: Asynchronous byzantine agreement protocols. *Information and Computation* **75**(2), 130–143 (1987). [https://doi.org/https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/https://doi.org/10.1016/0890-5401(87)90054-X), <https://www.sciencedirect.com/science/article/pii/089054018790054X>
11. Bracha, G., Toueg, S.: Resilient consensus protocols. In: Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing. p. 12–26. PODC ’83, Association for Computing Machinery, New York, NY, USA (1983). <https://doi.org/10.1145/800221.806706>, <https://doi.org/10.1145/800221.806706>
12. Cachin, C., Kursawe, K., Shvartsman, V.: Optimistic asynchronous byzantine agreement. In: Proceedings of the 24th IEEE Symposium on Reliable Distributed Systems. pp. 204–213 (2005)

13. Clement, A., Kapritsos, M., Lee, S., Wang, Y., Alvisi, L., Dahlin, M., Riche, T.: Upright cluster services. In: Proceedings of the 22nd ACM Symposium on Operating Systems Principles. pp. 277–290 (2009)
14. Constantinescu, A., Dufay, M., Ghinea, D., Wattenhofer, R.: Validity in Network-Agnostic Byzantine Agreement. In: Kowalski, D.R. (ed.) 39th International Symposium on Distributed Computing (DISC 2025). Leibniz International Proceedings in Informatics (LIPIcs), vol. 356, pp. 24:1–24:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2025). <https://doi.org/10.4230/LIPIcs.DISC.2025.24>, <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2025.24>
15. Constantinescu, A., Ghinea, D., Sliwinski, J., Wattenhofer, R.: Unifying partial synchrony. CoRR **abs/2405.10249** (2024). <https://doi.org/10.48550/ARXIV.2405.10249>, <https://doi.org/10.48550/arXiv.2405.10249>
16. Constantinescu, A., Ghinea, D., Wattenhofer, R., Westermann, F.: Convex Consensus with Asynchronous Fallback. In: Alistarh, D. (ed.) 38th International Symposium on Distributed Computing (DISC 2024). Leibniz International Proceedings in Informatics (LIPIcs), vol. 319, pp. 15:1–15:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2024). <https://doi.org/10.4230/LIPIcs.DISC.2024.15>, <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2024.15>
17. Deligios, G., Erbes, M.M.: Closing the efficiency gap between synchronous and network-agnostic consensus. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part V. LNCS, vol. 14655, pp. 432–461. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58740-5_15
18. Dolev, D., Strong, H.R.: Authenticated algorithms for byzantine agreement. SIAM Journal on Computing **12**(4), 656–666 (1983)
19. Dwork, C., Lynch, N.A., Stockmeyer, L.: Consensus in the presence of partial synchrony. Journal of the ACM **35**(2), 288–323 (1988)
20. Ghinea, D., Liu-Zhang, C.D., Wattenhofer, R.: Multidimensional approximate agreement with asynchronous fallback. In: Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures. p. 141–151. SPAA ’23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3558481.3591105>, <https://doi.org/10.1145/3558481.3591105>
21. Giridharan, N., Abraham, I., Crooks, N., Nayak, K., Ren, L.: Granular Synchrony. In: Alistarh, D. (ed.) 38th International Symposium on Distributed Computing (DISC 2024). Leibniz International Proceedings in Informatics (LIPIcs), vol. 319, pp. 30:1–30:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2024). <https://doi.org/10.4230/LIPIcs.DISC.2024.30>, <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2024.30>
22. Golan Gueta, G., Abraham, I., Grossman, S., Malkhi, D., Pinkas, B., Reiter, M., Seredinschi, D.A., Tamir, O., Tomescu, A.: Sbft: A scalable and decentralized trust infrastructure. In: 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 568–580 (2019). <https://doi.org/10.1109/DSN.2019.00063>
23. Ji, X., Liu-Zhang, C.D., Song, Y.: Perfectly secure network-agnostic MPC comes for free. In: Daemen, J., Thomé, E. (eds.) Advances in Cryptology – EUROCRYPT 2026. pp. 337–365. Springer Nature Switzerland, Cham (2026)
24. Lynch, N.A.: Distributed Algorithms. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1996)

25. Malkhi, D., Nayak, K., Ren, L.: Flexible byzantine fault tolerance. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019. pp. 1041–1053. ACM Press (Nov 2019). <https://doi.org/10.1145/3319535.3354225>
26. Momose, A., Ren, L.: Multi-threshold byzantine fault tolerance. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021. pp. 1686–1699. ACM Press (Nov 2021). <https://doi.org/10.1145/3460120.3484554>
27. Parvedy, P., Raynal, M.: Uniform agreement despite process omission failures. In: Proceedings International Parallel and Distributed Processing Symposium. pp. 7 pp.– (2003). <https://doi.org/10.1109/IPDPS.2003.1213388>
28. Shi, E.: Foundations of distributed consensus and blockchains (2020), <https://www.distributedconsensus.net>, book manuscript

A Additional Comments on Defining Agreement

There are two slight subtleties in choosing an appropriate definition. The first is that protocols in the crash fault model often aim for *uniform agreement*, which requires agreement among *all* parties that decide an output. This property is not achievable in the presence of Byzantine faults, so Byzantine fault-tolerant agreement protocols settle for *non-uniform agreement*, which requires agreement only among non-faulty parties. Thus, in choosing our target definition, we must choose whether to settle for non-uniform agreement in the crash model or insist on uniform agreement. In this work, we aim for the ‘best of both worlds’: uniform agreement in the presence of crash faults and non-uniform agreement in the presence of Byzantine faults.

The second subtlety relates to validity, i.e., which outputs are considered valid given parties’ initial inputs. Two of the most common notions are *strong validity* and *weak validity*. Strong validity requires that if all non-faulty parties input the same value, then all non-faulty parties output that value, whereas weak validity (sometimes called ‘honest input’ validity) requires that any value output by a benign party must have been the input of a benign party. Strong validity is only achievable for $t < n/2$ faults [27], even for crash faults, and even in a synchronous network. For this reason, strong validity is not the default choice for settings where weaker notions of validity are achievable for $t \geq n/2$ faults (e.g. crash fault-tolerant protocols under synchrony). With this in mind, we define a slightly non-standard validity property, *benign-unanimous validity*. Note that in the presence of crash faults, benign-unanimous validity and weak validity are equivalent for binary agreement (but not multivalued agreement).

As a final note, nontriviality is not a standard property in the consensus literature, but it is weaker than the termination property when there is at least one correct party, so it does not change the problem; we find it convenient for resolving some edge cases.

B Deferred Proofs

B.1 Impossibility for $\mathbf{SC} \cup \mathbf{SB}$

Proof (Proof of Theorem 3). Definition 3 is trivially infeasible for $t_c \geq n$ because no protocol can guarantee nontriviality when $t_c = n$.

Standard (non-uniform and strongly valid) Byzantine agreement is known to be impossible for $2t_b \geq n$ (even assuming PKI). In the presence of Byzantine faults, Definition 3 has precisely the same requirements as standard Byzantine agreement, because the set of benign (non-Byzantine) parties is exactly the set of correct parties. Thus, the standard impossibility result for synchronous Byzantine agreement immediately extends to any $\mathbf{SB} \cup \mathbf{X}$ model pair.

B.2 Feasibility for $\mathbf{AC} \cup \mathbf{AB}$

Following [28], call a round $R \geq 2$ *lucky* if there does not and will never exist a PJ round- $(R-1)$ vote for $1 - \text{coinflip}(R)$. In other words, the coin does not conflict with any PJ previous-round vote, so they are all for the coin's value or for $\perp$.

Lemma 8. *If R is lucky then all parties terminate by round $R + 2$, even if t_c parties are Byzantine.*

Proof. Let $c = \text{coinflip}(R)$. In this case, all correct parties cast round- R prevotes for c , either justified by at least one round- $(R-1)$ vote for c , or by $n - t_c$ round- $(R-1)$ votes for $\perp$ (and based on the coin). Furthermore, no party will certify a prevote for $1 - c$ because the necessary justification cannot exist. Thus, every correct party eventually certifies $n - t_c$ prevotes for c , and casts a round- R vote for c . In round $R + 1$, then, there are $n - t_c$ votes for c (and certainly at most $t_c < n - t_c$ votes for $1 - c$, though in fact there cannot be any), so all correct parties output c at the start of round $R + 1$. $\square$

Lemma 9. *Fix a round $R \geq 2$. The probability that at least one of the rounds $R, R + 1$ is lucky is at least 50%, no matter the events of earlier rounds.*

Proof. There are two possibilities, both of which lead to one of these rounds being lucky with 50% probability.

Case 1: At the time the round- R coin is flipped, when the adversary learns its value c , some correct party has already certified a vote for a bit $b \in \{0, 1\}$. Then, by the unpredictability of the coin, there is a 50% chance $c = b$, making R lucky.

Case 2: At the time the round- R coin is flipped, all votes that have been received and certified are for $\perp$. Then, if a benign party p_i is one of the $n - t_c$ parties that flipped the coin, its vote set V'_{R-1} is unanimous for $\perp$. Therefore p_i will, once the coin is flipped, not cast a round- R prevote for $1 - c$. This means there are at least $n - t_c - t_b$ correct parties that will not cast one for $1 - c$.

Now, a round- R vote for $1 - c$ would require $n - t_c$ round- R prevotes for $1 - c$, but only $t_c + t_b < n - t_c$ such prevotes from distinct parties can exist. By

the unpredictability of the coin and the fact that at this time parties only just called $\text{coinflip}(R)$ and so none has gotten to $\text{coinflip}(R+1)$, there is a 50% chance that $\text{coinflip}(R+1)$ is going to be c . Thus, there is a 50% chance that round $R+1$ will be lucky. $\square$

B.3 Feasibility for $\text{SB} \cup \text{AC}$

We now go through the proof of feasibility for model pair $\text{SB} \cup \text{AC}$ “the long way”. Throughout, all claims about protocol properties assume the protocol is being run in model pair $\text{SB} \cup \text{AC}$.

Lemma 10. (*Credibility*) *In an execution of Π_{VAC} , if a correct party sends a message, all correct parties eventually certify it (or terminate first).*

Proof. The correct party p_i that sends a message does so on the basis of some messages that p_i certifies, which it forwards when it receives them; p_i certifies these on the basis of some other p_i -certified messages which p_i also receives and forwards, and so on. All of these messages eventually arrive at all other non-terminating correct parties, so they all eventually certify all of them.

Lemma 11. (*AC validity*) *In an execution of Π_{VAC} in which all parties receive input v , if at most h parties are crash faulty and all others are correct, then all correct parties output v .*

Proof. There are $\geq n-h$ votes from correct parties for v sent in round-0, and no votes for $1-v$ since all parties are benign. Therefore, all correct parties receive and certify $\geq n-h$ round-0 votes for v . Once a correct party p_i has waited Δ time, a waiting period that eventually ends, p_i casts a prevote in round 1 for v . Again, no party casts a prevote for $1-v$. All correct parties certify at least $n-h$ round-1 prevotes for v , so (after another finite-length wait) cast a round-1 vote for v . So all correct parties certify $n-h$ votes for v in round 2. Then they all decide v at the start of round 2 and ignore the coin flip. $\square$

Let F be the first round for which any correct party eventually certifies a termination message marked for round F . The next lemma shows consistency through round F .

Lemma 12. (*Intra-round consistency*) *In an execution of Π_{VAC} , if up to h parties crash and other parties are correct, then the following hold for any R with $1 \leq R \leq F$:*

- (a) *Every correct party prevotes and votes exactly once in round R .*
- (b) *There is not both a PJ round- R vote for 0 and a PJ round- R vote for 1.*

Proof. We induct on R . In round 1, every correct party receives at least $n-h$ round-0 votes, all of which are immediately certified, so every correct party prevotes exactly once. Since at least $n-h$ parties are correct, every correct party receives (and certifies, by 10) enough prevotes to vote, and does so once.

Algorithm 3: Π_{VAC} , from the perspective of party p_i with input $v_i \in \{0, 1\}$ (numbered lines are different from Algorithm 2)

Round 0.
 └ Multicast $\langle \text{vote}, r = 0, v_i \rangle$.

Round 1.
 1 **Upon** certifying a set V'_0 of $n - h$ round-0 votes signed by
 $V_0 = \{n - h \text{ parties}\}$ **and** Δ time has passed since last
 vote/prevote:
 └ $b \leftarrow$ the majority bit in V'_0 (breaking a tie with 0).
 └ Multicast $\langle \text{prevote}, r = 1, b, V_0 \rangle_i$.

2 **Upon** certifying a set U'_1 of $n - h$ round-1 prevotes signed by
 $U_1 = \{n - h \text{ parties}\}$ **and** Δ time has passed since last
 vote/prevote:
 3 └ **if** $n - h$ prevotes in U'_1 are for some b' **then**
 └ $v \leftarrow b'$.
 else
 └ $v \leftarrow \perp$.
 └ Multicast $\langle \text{vote}, r = 1, v, U_1 \rangle_i$.

Round R , with $2 \leq R$.
 Complete all earlier rounds first.

4 **Upon** certifying a set V'_{R-1} of $n - h$ round- $(R - 1)$ votes signed by
 $V_{R-1} = \{n - h \text{ parties}\}$ **and** Δ time has passed since last
 vote/prevote:
 └ $c \leftarrow \text{coinflip}(R)$.
 └ **if** V'_{R-1} contains a vote for a bit $b \in \{0, 1\}$ **then**
 └ Multicast $\langle \text{prevote}, r = R, b, V_{R-1} \rangle_i$.
 └ **else** (if V'_{R-1} is unanimous for $\perp$)
 └ Multicast $\langle \text{prevote}, r = R, c, V_{R-1} \rangle_i$.

5 **Upon** certifying a set U'_R of $n - h$ round- R prevotes signed by
 $U_R = \{n - h \text{ parties}\}$ **and** Δ time has passed since last
 vote/prevote:
 6 └ **if** $n - h$ prevotes in U'_R are for some b' **then**
 └ $v \leftarrow b'$.
 else
 └ $v \leftarrow \perp$.
 └ Multicast $\langle \text{vote}, r = R, v, U_R \rangle_i$.

Upon there existing a round $R \geq 1$ and a bit $b \in \{0, 1\}$ such that there are at
 least $n - h$ certified votes from round R on bit b , signed by a set T_R of $n - h$
 distinct parties:
 └ Complete execution as above for all rounds up to and including R .
 └ Multicast $\langle \text{terminate}, r = R, b, T_R \rangle_i$.
 └ Output b and terminate.

Thus, (a) holds in round 1. Next, if there is a PJ round-1 vote for 0 that cites a

Algorithm 4: $\Pi_{\text{SBUAC}}^{t_c}$, from the perspective of party p_i with input $v_i \in \{0, 1\}$

1. **Phase 1:** Run an instance (referred to as the i th instance) of Dolev-Strong (DS) broadcast as the sender, broadcasting v_i .
 2. After $(\lceil n/2 \rceil + 1) \cdot \Delta$ time passes, consider all n instances of DS complete. Take $\perp$ to be the default output to the j th instance if the extracted set for that instance is not of size 1 at this time.
 3. There is now a sequence of n outputs from the DS instances. Let v be the majority non- $\perp$ output, breaking a tie with 0.
 4. **Phase 2:** Run protocol Π_{VAC} (Algorithm 3) with input v .
-

set of $n - h$ parties and a PJ round-1 vote for 1 that cites a set of $n - h$ parties, these two sets intersect in at least $n - 2h \geq 1$ parties, so some party is in the intersection. However, every party is benign, and no benign party prevotes for both 0 and 1, by (a). Thus, (b) holds for $R = 1$.

Now let $R > 1$ and suppose (a) and (b) hold for round $R - 1$. Again, each correct party prevotes at least once, since it eventually certifies enough votes from correct parties even if the incorrect parties fail to send messages. Note that this relies on a correct party not terminating, and none does at this round: terminating based on round- F votes only happens after round F , while terminating based on votes from a round $< F$ would lead to a termination message being sent and certified unless all correct parties terminate in which case no round- F message would be PJ. Correct parties do not prevote twice, because that would require PJ round- $(R - 1)$ votes for both 0 and 1, which cannot exist by the inductive hypothesis. This shows point (a). Point (b) holds by the same reasoning as given above for round 1. $\square$

The arguments above do not work in rounds $\geq F + 1$. This is because termination messages are only justified by votes from one round, but can be used in later rounds as both votes and prevotes; as such, the overlapping argument above does not hold. The following lemma shows, however, that in rounds after F the behavior of Π_{VAC} is even more constrained.

Lemma 13. (*Consistency*) *In an execution of Π_{VAC} in which any correct party certifies a termination message m for round F , all correct parties output b by the end of round $F + 2$.*

Proof. First, note that round- F votes cannot justify both a termination message for 0 and for 1, by Lemma 12, which holds in round F . Next, a termination message must be justified by at least $n - h$ round- F votes, which overlaps with any set of $n - h$ round- F votes by at least $n - 2h \geq 1$ votes; thus, every correct and yet-to-terminate party certifies, at the start of round $F + 1$, at least one vote cited by m . Since all parties are benign, and benign parties do not equivocate, such a vote must be for b . As such, each correct party casts a prevote (possibly

through a termination message) for b in round $F + 1$. Also, no correct party certifies any prevote for $1 - b$ since justifying one necessitates one of

- A vote for $1 - b$. This is impossible by Lemma 12.
- $n - h$ PJ $\perp$ votes from distinct parties. This is impossible because there are at least $n - h$ benign parties that vote for b , so at most $h < n - h$ votes for $\perp$.

It follows that all correct parties vote for b in round $F + 1$ (possibly through a termination message), so all remaining correct parties output b at the start of round $F + 2$. $\square$

We now show that parties are unlikely to continue through infinitely many rounds. Following [28], let a round $R \geq 2$ be *lucky* if there does not and will never exist a PJ round- $(R - 1)$ vote for $1 - \text{CoinFlip}(R)$. In other words, the coin does not conflict with any PJ previous-round vote, so they are all for the coin's value or for $\perp$.

Lemma 14. *In an execution of Π_{VAC} in which at most h parties are crash faulty and all others are correct, if R is lucky then all parties terminate by round $R + 2$.*

Proof. Let $c = \text{CoinFlip}(R)$. In this case, all correct parties cast round- R prevotes for c , either justified by at least one round- $(R - 1)$ vote for c , or by $n - h$ round- $(R - 1)$ votes for $\perp$ (and based on the coin). Furthermore, no party will certify a prevote for $1 - c$ because the necessary justification cannot exist. Thus, every correct party eventually certifies $n - h$ prevotes for c , and casts a round- R vote for c . In round $R + 1$, then, there are $n - h$ votes for c (and certainly at most $h < n - h$ votes for $1 - c$, though in fact there cannot be any), so all correct parties output c at the start of round $R + 1$. $\square$

Lemma 15. *Fix a round $R \geq 2$. The probability that at least one of the rounds $R, R + 1$ is lucky is at least 50%, no matter the events of earlier rounds.*

Proof. There are two possibilities, both of which lead to one of these rounds being lucky with 50% probability.

Case 1: At the time the round- R coin is flipped, when the adversary learns its value c , some correct party has already certified a vote for a bit $b \in \{0, 1\}$. Then, by the unpredictability of the coin, there is a 50% chance $c = b$, making R lucky.

Case 2: At the time the round- R coin is flipped, all votes that have been received and certified are for $\perp$. Then, if a benign party p_i is one of the $n - h$ parties that flipped the coin, its vote set V'_{R-1} is unanimous for $\perp$. Therefore p_i will, once the coin is flipped, not cast a round- R prevote for $1 - c$. Thus, there are at least $n - h$ correct parties that will not cast a prevote for $1 - c$.

Now, a round- R vote for $1 - c$ would require $n - h$ round- R prevotes for $1 - c$, but only $h < n - h$ such prevotes from distinct parties can exist. By the unpredictability of the coin and the fact that at this time parties only just called

$\text{CoinFlip}(R)$ and so none has gotten to $\text{CoinFlip}(R+1)$, there is a 50% chance that $\text{CoinFlip}(R+1)$ is going to be c . Thus, there is a 50% chance that round $R+1$ will be lucky. $\square$

Thus ends “the long way”.

Lemma 16. (*Validity*) *In an synchronous execution of Π_{VAC} in which less than $n/2$ parties are faulty, and all benign parties receive input v , all correct parties output v by the end of round 2.*

Proof. All correct parties prevote v in round 0. At the start of round 1, all correct parties receive all others’ prevotes, because they wait an additional Δ time. Since $< n/2$ parties are Byzantine, all correct parties prevote v in round 1. Again, all correct parties receive all of these prevotes within Δ waiting. Also, all of these prevotes will be justified by a set containing every correct party, so each correct party will certify all of these prevotes, since it will have received messages from those $\geq n-h$ correct parties implying a majority for v . Therefore, in the second step of round 1 all correct parties have sufficiently many PJ prevotes for v to vote for v . Furthermore, casting a PJ vote for $1-v$ requires $n-h > n/2$ distinct parties to cast a round-1 prevote for $1-v$, but no benign party casts one, so this is impossible; as such, no Byzantine party can cast a PJ vote for $1-v$. Thus, at the start of round 2 all correct parties receive (due to synchrony) and certify (due to benign-majority prevote unanimity for v) more than $n/2$ votes for v and no votes for $1-v$. At this point all correct parties decide v and terminate. $\square$

Putting all the pieces together, we can now complete the proof of Theorem 5.

Proof (Proof of Theorem 5). Suppose the network is synchronous. The unconditional consistency of the Dolev-Strong protocol in synchrony shows that all correct parties end the first phase with the same sequence of n bits. Thus, all correct parties begin the second phase with the same bit as input. Since Π_{VAC} is valid under $< n/2$ Byzantine faults in synchrony, consistency is obtained, in addition to termination by 16. Also, if all correct parties begin with the same bit v , then by validity of DS, every correct party ends the first phase with $> n/2$ of the elements of its output sequence equal to v . Therefore, all correct parties enter the second phase with input v , and all decide v by validity of Π_{VAC} .

Otherwise, the network is asynchronous. Each party eventually exits the first phase, and by Lemmas 8 and 9, the second phase has termination with probability 1 under $t < n/2$ benign faults in asynchrony. Thus, Π_{VAC} achieves termination with probability 1.

Because Π_{VAC} is consistent in the presence of $t < n/2$ crash faults even in asynchrony, consistency of $\Pi_{\text{SBUAC}}^{t_c}$ is guaranteed. Also, if every correct party begins with input v , each must output $\perp$ or v from every instance of DS, as no message will ever be for bit $1-v$ when all parties are benign and start with input v . Furthermore, each party must output v from its own instance of DS. Therefore, the majority non- $\perp$ output will be v . So the validity of Π_{VAC} in asynchrony implies $\Pi_{\text{SBUAC}}^{t_c}$ also achieves validity in asynchrony. $\square$